

Guía del programador de historias de AFA Engine

Bienvenido a la guía oficial para crear historias interactivas con **AFA Engine**, un motor de aventuras gráficas diseñado para ser completamente personalizable. Esta guía está pensada para programadores con experiencia intermedia en JavaScript y te enseñará cómo estructurar tus historias usando exclusivamente las funciones de la API del motor.

Arquitectura general

AFA Engine se compone de los siguientes módulos:

Archivo	Rol principal
engine.js	Núcleo del motor. Maneja el estado del juego y ejecuta scripts.
api.js	API pública. Expone funciones para los scripts de historia.
renderer.js	Dibuja escenas y personajes en pantalla.
ui.js	Controla la interfaz de usuario y las interacciones visuales.
game.js	Punto de entrada del juego. Inicializa y lanza el motor.
data.js	Archivo de historia. Define personajes, escenas, objetos, ítems y diálogos.

Estructura de data.js

Tu historia se define en el objeto `storyData`. Este contiene:

◆ Inicialización

```
inits: {  
  storyName: 'Nombre de la historia',  
  player: 'Nombre del personaje jugador',  
  scene: 'Escena inicial'  
}
```

◆ Verbos

Define las acciones disponibles para el jugador. Cada verbo tiene:

- **label**: nombre interno
- **display**: texto visible en la UI
- **expects**: tipos de argumentos requeridos (`item`, `object`, `character`)
- **optional**: argumentos opcionales
- **connector / preposition**: conectores gramaticales

Ejemplo:

```
AGARRAR: {  
  label: 'Agarrar',  
  display: 'Agarrar',  
  expects: [['item', 'object']]  
}
```

◆ Personajes (characters)

Cada personaje tiene:

- name, alias, description
- inventory: array de ítems
- x, y, width, height: posición y tamaño
- image: ruta de imagen (opcional)
- playable: si puede ser jugador

◆ Objetos (objects)

Definidos por:

- name, description
- x, y, width, height
- Estados personalizados (isOpen, hasCarbon, etc.)
- exits: nombre de escena destino (si es una salida)

◆ Ítems (items)

Cada ítem tiene:

- name, description
- x, y, width, height, color
- canBePickedUp: si puede recogerse
- isHidden: si está oculto
- flags: marcas personalizadas

◆ Escenas (scenes)

Cada escena contiene:

- name
- background: colores de pared y piso
- objects: array de objetos presentes
- items: array de ítems presentes
- characters: array de personajes presentes

◆ Diálogos (dialogueMatrix)

Estructura de árbol de diálogo por personaje. Cada nodo tiene:

- **player**: frase del jugador
 - **npc**: respuesta del NPC
 - **leadsTo**: ID del siguiente nodo
 - **runScript**: nombre de script a ejecutar (opcional)
 - **setsAfaLocation**: marcador de historia (opcional)
-

Scripts de acción

Los scripts se definen como funciones dentro de `storyScripts` en `data.js`. Deben usar **únicamente funciones de `api.js`** para interactuar con el motor.

Resolución de nombres

Cuando el jugador ejecuta una acción, el motor busca el script en este orden:

1. `ACTOR_VERBO_ITEM_TARGET`
2. `ACTOR_VERBO_ITEM`
3. `ACTOR_VERBO`
4. `VERBO_ITEM_TARGET`
5. `VERBO_ITEM`
6. `default`

Todos los nombres se transforman a minúsculas.

Funciones disponibles en `api.js`

Aquí están las funciones que podés usar en tus scripts, agrupadas por categoría:

Estado del juego

```
getGameData()  
getGameState()  
getCurrentPlayer()  
getCurrentScene()
```

Personajes

```
addPlayer(name)  
removePlayer(name)  
changeActor(name, ui)  
getActor(name)  
actorHasItem(name, item)  
actorChangeScene(actor, destination)  
showActor(name)
```

Inventario

```
addItemToActor(character, item)
removeItemFromActor(character, item)
getInventory(character)
getItem(item)
dropItem(item)
switchItem(oldItem, newItem)
giveItemTo(item, character)
```

Ítems

```
getItemData(item)
moveItemToScene(item, scene)
setItemFlag(item, flag, value)
toggleItemFlag(item, flag)
isItemFlagSet(item, flag)
canItemBePickedUp(item)
isItemHidden(item)
```

Objetos

```
getObjectData(object)
setObjectState(object, state, value)
getObjectState(object, state)
isObjectInState(object, state, value)
```

Diálogos

```
startDialogue(listener, nodeID)
say(text)
abortDialogues()
```

Interfaz

```
hideUI()
showUI()
```

Tiempo

```
wait(frames) // espera en frames (~60fps)
setTimer(callback, ms)
clearTimer(timerID)
```

Scripts

```
runScript(name)
```

Ejemplo de script

```
storyScripts['pancho_agarrar_botella_fernet'] = () => {
  if (api.canItemBePickedUp('botella_fernet')) {
    api.getItem('botella_fernet');
    api.say("Pancho agarró el fernet. ¡Ahora sí empieza la fiesta!");
  } else {
    api.say("No podés agarrar eso todavía.");
  }
};
```



Buenas prácticas

- Usá `say()` para mostrar texto en pantalla.
 - Verificá existencia de ítems/personajes antes de operar.
 - Usá `getGameData()` para acceder a la estructura completa si necesitás inspeccionar algo.
 - Evitá modificar directamente `gameData` o `gameState`. Usá funciones API.
 - Usá `wait()` para pausas dramáticas o efectos de tiempo.
 - Mantené los nombres de scripts en minúsculas y sin espacios.
-



Funciones de `api.js` en AFA Engine

Estas funciones son la **API pública** del motor. Son las únicas que debés usar para escribir tus scripts en `data.js`. No modifiques directamente `gameData` ni `gameState`: usá estas funciones para mantener la integridad del motor.



Estado del juego

Función	Descripción
<code>getGameData()</code>	Devuelve el objeto completo <code>gameData</code> , que contiene personajes, escenas, ítems, objetos, etc. Útil para inspeccionar el mundo.
<code>getGameState()</code>	Devuelve el estado actual del juego (<code>gameState</code>), incluyendo el jugador activo, escena actual y acción en curso.
<code>getCurrentPlayer()</code>	Devuelve el nombre del personaje jugador actual.
<code>getCurrentScene()</code>	Devuelve el nombre de la escena actual.



Personajes

Función	Descripción
<code>addPlayer(name)</code>	Convierte a un personaje en jugable (<code>playable = true</code>) y actualiza el selector de personajes.
<code>removePlayer(name)</code>	Lo contrario: lo vuelve no jugable (<code>playable = false</code>).

Función	Descripción
<code>changeActor(name, ui)</code>	Cambia el jugador activo al personaje indicado y actualiza la UI.
<code>getActor(name)</code>	Devuelve el objeto del personaje por su nombre.
<code>actorHasItem(name, item)</code>	Verifica si el personaje tiene un ítem en su inventario.
<code>actorChangeScene(actor, destination)</code>	Mueve al personaje a otra escena. Si es el jugador actual, también cambia la escena activa.
<code>showActor(name)</code>	Cambia la escena actual a aquella donde se encuentra el personaje indicado.

Inventario

Función	Descripción
<code>addItemToActor(character, item)</code>	Agrega un ítem al inventario del personaje. Actualiza la UI si es el jugador actual.
<code>removeItemFromActor(character, item)</code>	Quita un ítem del inventario del personaje. También actualiza la UI si corresponde.
<code>getInventory(character)</code>	Devuelve el inventario del personaje como array.
<code>getItem(item)</code>	El jugador recoge un ítem del escenario actual. Lo quita de la escena y lo agrega al inventario.
<code>dropItem(item)</code>	El jugador suelta un ítem en la escena actual. Lo quita del inventario y lo agrega a la escena.
<code>switchItem(oldItem, newItem)</code>	Reemplaza un ítem por otro, ya sea en el inventario o en la escena.
<code>giveItemTo(item, character)</code>	El jugador le da un ítem a otro personaje. Lo quita de su inventario y lo agrega al del destinatario.

Ítems

Función	Descripción
<code>getItemData(item)</code>	Devuelve el objeto del ítem por su nombre.
<code>moveItemToScene(item, scene)</code>	Mueve un ítem a una escena específica. Lo quita de otras escenas si es necesario.
<code>setItemFlag(item, flag, value)</code>	Establece una marca (flag) en el ítem.
<code>toggleItemFlag(item, flag)</code>	Alterna el valor de una marca booleana.
<code>isItemFlagSet(item, flag)</code>	Verifica si una marca está activa.
<code>canItemBePickedUp(item)</code>	Devuelve <code>true</code> si el ítem puede recogerse (<code>canBePickedUp</code>).
<code>isItemHidden(item)</code>	Devuelve <code>true</code> si el ítem está oculto (<code>isHidden</code>).

Objetos

Función	Descripción
<code>getObjectData(object)</code>	Devuelve el objeto por su nombre.
<code>setObjectState(object, state, value)</code>	Cambia el estado de un objeto (por ejemplo, <code>isOpen = true</code>). Redibuja si está en la escena actual.
<code>getObjectState(object, state)</code>	Devuelve el valor de un estado del objeto.
<code>isObjectInState(object, state, value)</code>	Verifica si el objeto está en un estado específico.

Diálogos

Función	Descripción
<code>startDialogue(listener, nodeID)</code>	Inicia un diálogo entre el jugador actual y el personaje indicado, comenzando en el nodo especificado.
<code>say(text)</code>	Muestra una línea de diálogo en pantalla con velocidad proporcional a la longitud del texto.
<code>abortDialogues()</code>	(Vacía por ahora) Cancelaría diálogos activos.

Interfaz

Función	Descripción
<code>hideUI()</code>	Oculto todos los elementos de la interfaz gráfica.
<code>showUI()</code>	Muestra todos los elementos de la interfaz gráfica.

Tiempo

Función	Descripción
<code>wait(frames)</code>	Pausa la ejecución del script por una cantidad de frames (~60 por segundo). Devuelve una promesa.
<code>setTimer(callback, ms)</code>	Ejecuta una función después de un tiempo en milisegundos.
<code>clearTimer(timerID)</code>	Cancela un temporizador. (No implementado aún)

Scripts

Función	Descripción
<code>runScript(name)</code>	Ejecuta un script de historia por su nombre.
<code>runActionScript(name)</code>	Ejecuta directamente el script desde <code>storyScripts</code> . Usado internamente por el motor.

¿Qué es el script "default"?

Es un **script de respaldo** que se ejecuta **cuando el jugador realiza una acción que no tiene un script específico definido** en storyScripts.

¿Cuándo se ejecuta?

Cuando el jugador selecciona un verbo y uno o dos elementos (ítem, objeto, personaje), el motor intenta encontrar un script que coincida con esa combinación. El orden de búsqueda es:

1. ACTOR_VERBO_ITEM_TARGET
2. ACTOR_VERBO_ITEM
3. ACTOR_VERBO
4. VERBO_ITEM_TARGET
5. VERBO_ITEM
6. **default**

Si **ninguna de las combinaciones anteriores existe** en storyScripts, se ejecuta el script "default".

¿Para qué sirve?

- Evita que el juego quede sin respuesta ante acciones no previstas.
 - Permite mostrar un mensaje genérico como “No pasa nada” o “No podés hacer eso”.
 - Puede usarse para registrar intentos inválidos, dar pistas o mantener la inmersión.
-

Ejemplo de implementación

En tu data.js, podés definirlo así:

```
storyScripts['default'] = () => {  
  api.say("Esa acción no tiene efecto.");  
};
```

También podés hacerlo más sofisticado:

```
storyScripts['default'] = () => {  
  const verb = api.getState().actionState.verb;  
  api.say(`No podés usar "${verb.toLowerCase()}" en este contexto.`);  
};
```

Buenas prácticas

- Siempre definí un "default" para evitar silencios incómodos.

- Usalo para reforzar el tono de tu historia (humor, misterio, etc.).
 - Podés usarlo como herramienta de debugging para detectar combinaciones no cubiertas.
-



Ejemplo de script "default" con lógica por verbo

Este script se ejecuta cuando no hay un script específico para la acción del jugador. En lugar de responder siempre lo mismo, analiza el verbo seleccionado y da una respuesta contextual.

```
storyScripts['default'] = () => {
  const state = api.getState();
  const verb = state.actionState.verb;
  const item = state.actionState.item.key;
  const target = state.actionState.target.key;

  switch (verb) {
    case 'AGARRAR':
      api.say(item ? `No podés agarrar "${item}".` : "¿Qué querés agarrar?");
      break;

    case 'USAR':
      if (item && target) {
        api.say(`No pasa nada al usar "${item}" con "${target}".`);
      } else if (item) {
        api.say(`¿Con qué querés usar "${item}"?`);
      } else {
        api.say("¿Qué querés usar?");
      }
      break;

    case 'DAR':
      if (item && target) {
        api.say(`No podés darle "${item}" a "${target}".`);
      } else {
        api.say("¿Qué querés dar y a quién?");
      }
      break;

    case 'HABLAR':
      api.say(target ? `No hay nada que decirle a "${target}".` : "¿Con quién querés hablar?");
      break;

    case 'MIRAR':
      api.say(target ? `No ves nada especial en "${target}".` : "¿Qué querés mirar?");
      break;

    case 'ABRIR':
      api.say(target ? `"${target}" no se puede abrir.` : "¿Qué querés
```

```
abrir?");
    break;

    case 'CERRAR':
        api.say(target ? `${target}` no se puede cerrar.` : "¿Qué querés cerrar?");
        break;

    case 'IR':
        api.say(target ? `No podés ir hacia "${target}`.` : "¿A dónde querés ir?");
        break;

    default:
        api.say("Esa acción no tiene efecto.");
        break;
}
};
```

Ventajas de este enfoque

- Da respuestas más naturales y temáticas según el verbo.
 - Ayuda al jugador a entender qué falta o por qué no funciona.
 - Podés personalizar cada caso con humor, pistas o ambientación.
-